

Guía 6: Laboratorio de vulnerabilidades OWASP Top 10:2025 (A01, A02, A05, A07)

Cobos, Dilan Steven

Rodríguez Ángel, Santiago

Alarcón Ospina, Víctor Alfonso

Universidad Manuela Beltrán

Ingeniería de Software

Facultad de Ingeniería

Desarrollo de Software Seguro

Bogotá, Colombia

2026

Tabla de contenido

Introducción 6
Taller 1. A01:2025 Broken Access Control (SSRF) 7
Nombre de la actividad 7
Vulnerabilidad asociada 7
Descripción del entorno 7
Objetivo del reto 7
Instrucciones 8
Resultados esperados 10
Criterios de éxito 10
Medidas de mitigación 10
Preguntas de análisis 11
Taller 2. A02:2025 Security Misconfiguration 12
Nombre de la actividad 12
Vulnerabilidad asociada 12
Descripción del entorno 12
Objetivo del reto 12
Instrucciones 12
Resultados esperados 17
Criterios de éxito 17
Medidas de mitigación 17
Preguntas de análisis 18
Taller 3. A05:2025 Injection (XSS almacenado) 19
Nombre de la actividad 19
Vulnerabilidad asociada 19
Descripción del entorno 19
Objetivo del reto 19
Instrucciones 19
Resultados esperados 21
Criterios de éxito 21
Medidas de mitigación 21
Preguntas de análisis 21
Taller 4. A07:2025 Authentication Failures (JWT sin verificar) 22
Nombre de la actividad 22
Vulnerabilidad asociada 22
Descripción del entorno 22
Objetivo del reto 22
Instrucciones 22
Resultados esperados 25
Criterios de éxito 25
Medidas de mitigación 25
Preguntas de análisis 25

Resumen de los cuatro talleres..... 26
Anexo: cómo ejecutar el laboratorio..... 26
Referencias..... 26

Lista de tablas

Tabla 1 Vulnerabilidades trabajadas en la Guía 6..... 26

Lista de figuras

Figura 1 Explotación de SSRF: la vista previa vulnerable revela el servicio interno 9
Figura 2 Comparación: la versión vulnerable responde con los datos internos; la segura rechaza la URL..... 10
Figura 3 Versión vulnerable: el panel abre sin sesión y admin/admin entra como administrador 14
Figura 4 Versión segura: el panel exige sesión y admin/admin es rechazado 15
Figura 5 Versión segura: con la credencial válida se obtiene una sesión y el panel responde 16
Figura 6 Comparación: la etiqueta se ejecuta como HTML en la versión vulnerable (aparece XSS ejecutado) y queda como texto en la segura 20
Figura 7 Preparación del ataque y comparación: el perfil vulnerable acepta el rol admin falsificado; el seguro lo rechaza por firma inválida..... 24

Introducción

Este documento corresponde a la Guía 6 de Desarrollo de Software Seguro. El grupo seleccionó cuatro vulnerabilidades del OWASP Top 10:2025 que no se repiten con los talleres ya entregados por otros grupos del curso: A01:2025 Broken Access Control, con enfoque en Server-Side Request Forgery (SSRF); A02:2025 Security Misconfiguration; A05:2025 Injection, con enfoque en Cross-Site Scripting (XSS) almacenado; y A07:2025 Authentication Failures, con enfoque en un token de sesión (JWT) cuya firma no se verifica.

Para los cuatro talleres se construyó un laboratorio propio en Node.js y Express que corre únicamente en el equipo local (127.0.0.1), sin depender de un servicio externo. Cada módulo expone dos versiones del mismo flujo: una **vulnerable**, que reproduce el defecto descrito por OWASP, y una **segura**, que aplica la mitigación correspondiente sobre el mismo flujo. Esto permite comparar en vivo, ante el docente, el antes y el después de cada corrección.

Todas las capturas de este documento corresponden a ejecuciones reales del laboratorio, realizadas el 17 de octubre de 2026 en el navegador Microsoft Edge, con clics reales sobre cada botón y contra el servidor local. Cada respuesta muestra su código HTTP y un veredicto: «El ataque funcionó» en la versión vulnerable o «El ataque fue bloqueado» en la segura. El código fuente completo del laboratorio (carpeta **lab/**) se entrega junto con este documento.

Taller 1. A01:2025 Broken Access Control (SSRF)

Nombre de la actividad

La imagen que pidió el servidor: explotando un Server-Side Request Forgery.

Vulnerabilidad asociada

A01:2025 Broken Access Control. OWASP mantiene esta categoría en el puesto #1 del ranking: el 100 % de las aplicaciones probadas presentó alguna forma de control de acceso roto. Entre los CWE señalados explícitamente está **CWE-918 Server-Side Request Forgery (SSRF)**, que es el enfoque de este taller (OWASP Top 10:2025, A01).

Descripción del entorno

Aplicación Node.js/Express (módulo **modules/a01.js** del laboratorio) con una función de "vista previa de imagen": el usuario escribe una URL y el propio servidor hace esa petición por detrás, para luego mostrar la respuesta. Junto al servidor principal corre un segundo servicio HTTP en el puerto 4019, enlazado solo a 127.0.0.1, que simula un sistema interno (un "inventario") que nunca debería ser alcanzable desde un formulario público. Como medida de contención del propio laboratorio (y no como parte de la vulnerabilidad), el módulo solo permite peticiones http hacia 127.0.0.1 o localhost, de modo que el formulario no pueda usarse contra equipos ajenos.

Objetivo del reto

Demostrar que, cuando el servidor no valida qué URL puede pedir, un usuario puede hacer que ese servidor contacte un servicio interno que no está expuesto al público, y leer su respuesta. Luego comparar con una versión que solo acepta identificadores de una lista fija (allowlist), nunca una URL libre.

Instrucciones

Paso 1. Ejecutar **npm install && npm start** dentro de la carpeta **lab/** y abrir http://localhost:3000 en el navegador.

Paso 2. Abrir la pestaña **A01**. En el campo de la versión vulnerable ya aparece precargada la URL **http://127.0.0.1:4019/**, que es la del servicio interno.

Paso 3. Hacer clic en **Pedir vista previa**. Observar que el servidor responde 200 OK con el contenido del servicio interno (incluida una "clave de API" simulada), aunque nadie debería poder llegar hasta allí desde este formulario.

Paso 4. En la versión segura, hacer clic en **Probar con la URL interna**, que escribe esa misma URL en el campo de identificador y la envía. Verificar que la respuesta es 403 Prohibido: la versión segura solo acepta un valor de una lista fija (por ejemplo, **logo**), nunca una URL arbitraria.

Figura 1
Explotación de SSRF: la vista previa vulnerable revela el servicio interno



Nota. Elaboración propia.

Figura 2

Comparación: la versión vulnerable responde con los datos internos; la segura rechaza la URL



Nota. Elaboración propia.

Resultados esperados

La versión vulnerable devuelve el JSON del servicio interno (incluida la nota «Este endpoint nunca debería ser alcanzable desde fuera del servidor»). La versión segura devuelve el error «Identificador no permitido» para la misma URL.

Criterios de éxito

El estudiante demuestra que el servidor vulnerable contacta un servicio interno a partir de una URL controlada por el usuario, y que la versión segura rechaza esa misma URL por no estar en la lista permitida.

Medidas de mitigación

Código de la versión segura (módulo **modules/a01.js**):

```
app.get('/a01/safe/preview', (req, res) => {
  const id = req.query.id;
  if (id !== 'logo') {
    return res.status(403).json({ error: 'Identificador no permitido...' });
  }
```

```
  res.json({ solicitado: id, nota: 'Nunca se abre una URL que venga del usuario.' });
});
```

Nunca aceptar una URL completa escrita por el usuario para que el servidor la visite. Validar contra una lista fija de destinos permitidos (allowlist), nunca una lista de bloqueo. Cuando se necesite acceder a un recurso remoto, hacerlo por un identificador interno que el servidor resuelve, no por la URL literal. Segmentar la red para que los servicios internos no sean alcanzables ni siquiera si el filtro falla.

Preguntas de análisis

1. ¿Por qué una lista de bloqueo (bloquear "127.0.0.1", "localhost", etc.) es más fácil de evadir que una lista de permitidos? 2. Si el servicio interno del taller fuera, en una empresa real, el panel de administración de la nube (por ejemplo, el servicio de metadatos de un proveedor cloud), ¿qué información sensible podría exponer un SSRF como este?

Taller 2. A02:2025 Security Misconfiguration

Nombre de la actividad

admin / admin: el panel que nadie terminó de configurar.

Vulnerabilidad asociada

A02:2025 Security Misconfiguration. Subió del puesto #5 al #2; el 100 % de las aplicaciones probadas presentó alguna forma de mala configuración. OWASP describe explícitamente dos síntomas que este taller reproduce: **cuentas por defecto con su contraseña sin cambiar** y **mensajes de error que revelan trazas de pila o detalles internos** (OWASP Top 10:2025, A02).

Descripción del entorno

Módulo **modules/a02.js** del laboratorio: un panel de administración simulado. La versión vulnerable no exige sesión para leer la configuración del panel, conserva credenciales de fábrica (**admin / admin**) y, si el inicio de sesión falla, devuelve un error con una traza interna simulada. La versión segura no tiene credenciales de fábrica: guarda la contraseña como un hash scrypt con sal, emite un identificador de sesión aleatorio al iniciar sesión, exige ese identificador para leer el panel y responde con un mensaje genérico cuando las credenciales fallan.

Objetivo del reto

Demostrar que una ruta administrativa sin control de sesión, combinada con credenciales de fábrica nunca cambiadas, permite leer la configuración interna de la aplicación y entrar como administrador sin explotar ningún error de programación, solo una mala configuración.

Instrucciones

Paso 1. Con el laboratorio corriendo, abrir la pestaña **A02**.

Paso 2. En la versión vulnerable, hacer clic en **Abrir panel (sin sesión)**. Observar que el panel responde sin pedir ninguna credencial, y expone la configuración interna, incluidas las credenciales de fábrica.

Paso 3. Hacer clic en **Entrar con admin/admin**. Verificar que el inicio de sesión es exitoso y el rol devuelto es **administrador**.

Paso 4. Hacer clic en **Entrar con clave incorrecta** y observar que el error revela el nombre de una tabla y una traza de archivo y línea, información que nunca debería llegar al usuario.

Paso 5. En la versión segura, hacer clic en **Abrir panel (sin sesión)** y en **Probar admin/admin**. Verificar que el panel exige una sesión y que las credenciales de fábrica son rechazadas con un mensaje genérico, sin ningún dato interno.

Paso 6. Hacer clic en **Entrar con credencial válida**: el servidor devuelve un identificador de sesión. Luego hacer clic en **Abrir panel (con sesión)** y verificar que ahora el panel responde, sin ningún dato sensible.

Figura 3

Versión vulnerable: el panel abre sin sesión y admin/admin entra como administrador



Nota. Elaboración propia.

Figura 4

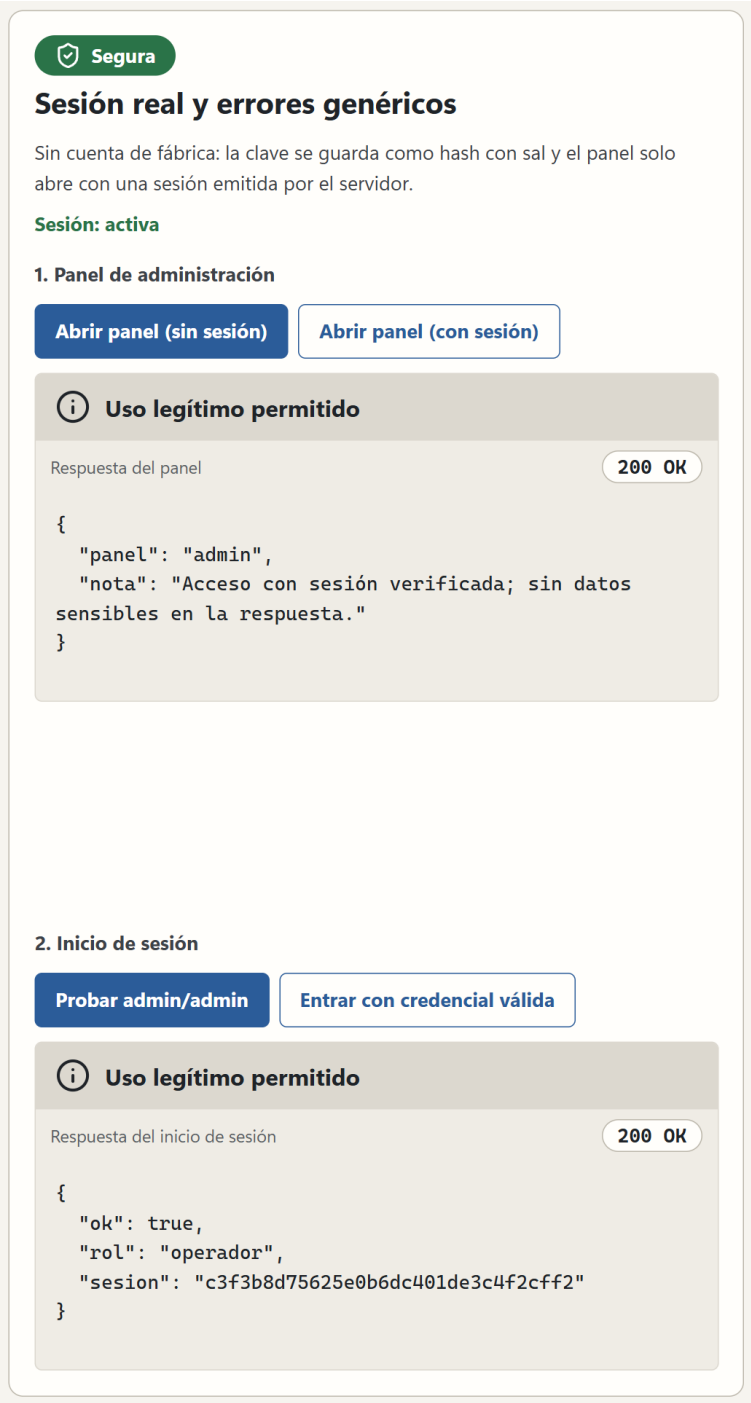
Versión segura: el panel exige sesión y admin/admin es rechazado



Nota. Elaboración propia.

Figura 5

Versión segura: con la credencial válida se obtiene una sesión y el panel responde



Nota. Elaboración propia.

Resultados esperados

La versión vulnerable entrega el panel sin sesión y acepta admin/admin con rol administrador. La versión segura devuelve «Se requiere sesión válida» para el panel sin sesión y «Credenciales inválidas» para admin/admin, sin ningún dato interno; con la credencial válida entrega una sesión y el panel responde.

Criterios de éxito

El estudiante demuestra el acceso al panel sin sesión, el inicio de sesión con credenciales de fábrica, y la traza interna expuesta en el error; y muestra que la versión segura bloquea las tres cosas.

Medidas de mitigación

Código de la versión segura (módulo **modules/a02.js**):

```
const STORED_HASH = crypto.scryptSync('unaClaveLarga123', SALT, 32); // nunca en claro
app.post('/a02/safe/login', (req, res) => {
  const { user, pass } = req.body || {};
  if (user === SAFE_USER && checkPassword(pass)) {
    const sesion = crypto.randomBytes(16).toString('hex');
    sessions.add(sesion);
    return res.json({ ok: true, rol: 'operador', sesion });
  }
  res.status(401).json({ ok: false, error: 'Credenciales inválidas.' });
});
app.get('/a02/safe/admin-panel', (req, res) => {
  if (!sessions.has(req.header('x-session'))) return res.status(401).json({ error: 'Se
requiere sesión.' });
  res.json({ panel: 'admin', nota: 'Sin datos sensibles.' });
});
```

Eliminar toda credencial de fábrica antes de desplegar, en especial para cuentas de administrador. Proteger cada ruta administrativa con verificación de sesión y de rol, no solo con su nombre poco conocido. Configurar manejo centralizado de errores que devuelva mensajes genéricos al cliente y registre el detalle real solo en el servidor. Usar el mismo proceso de endurecimiento (hardening) y las mismas credenciales por entorno distintas para desarrollo, pruebas y producción.

Preguntas de análisis

1. ¿Por qué "ocultar" la ruta del panel (no enlazarla desde el menú) no es una mitigación real?

2. ¿Qué información de la traza de error vulnerable usaría un atacante para planear su siguiente paso?

Taller 3. A05:2025 Injection (XSS almacenado)

Nombre de la actividad

Un comentario que no era solo texto: XSS almacenado en un libro de comentarios.

Vulnerabilidad asociada

A05:2025 Injection. El 100 % de las aplicaciones probadas se evaluó para alguna forma de inyección, y esta categoría tiene el mayor número de CVE de todo el Top 10:2025. OWASP identifica dentro de ella a **CWE-79 Improper Neutralization of Input During Web Page Generation (Cross-Site Scripting)**, con más de 30 000 CVE asociados, que es el enfoque de este taller (OWASP Top 10:2025, A05).

Descripción del entorno

Módulo **modules/a05.js** del laboratorio: un libro de comentarios muy simple. La versión vulnerable guarda el comentario y lo devuelve tal cual; la interfaz lo inserta en la página con **innerHTML**, por lo que cualquier etiqueta HTML dentro del comentario se ejecuta en el navegador de quien lo lee. La versión segura escapa los caracteres **< > & " '** antes de devolver el comentario, así que el navegador lo muestra como texto, nunca como código.

Objetivo del reto

Demostrar que un comentario con una etiqueta **<img>** con un manejador **onerror** se ejecuta como HTML real en la versión vulnerable (el navegador intenta cargar la imagen y dispara el evento), y que la misma entrada queda como texto inofensivo en la versión segura.

Instrucciones

Paso 1. Con el laboratorio corriendo, abrir la pestaña **A05**.

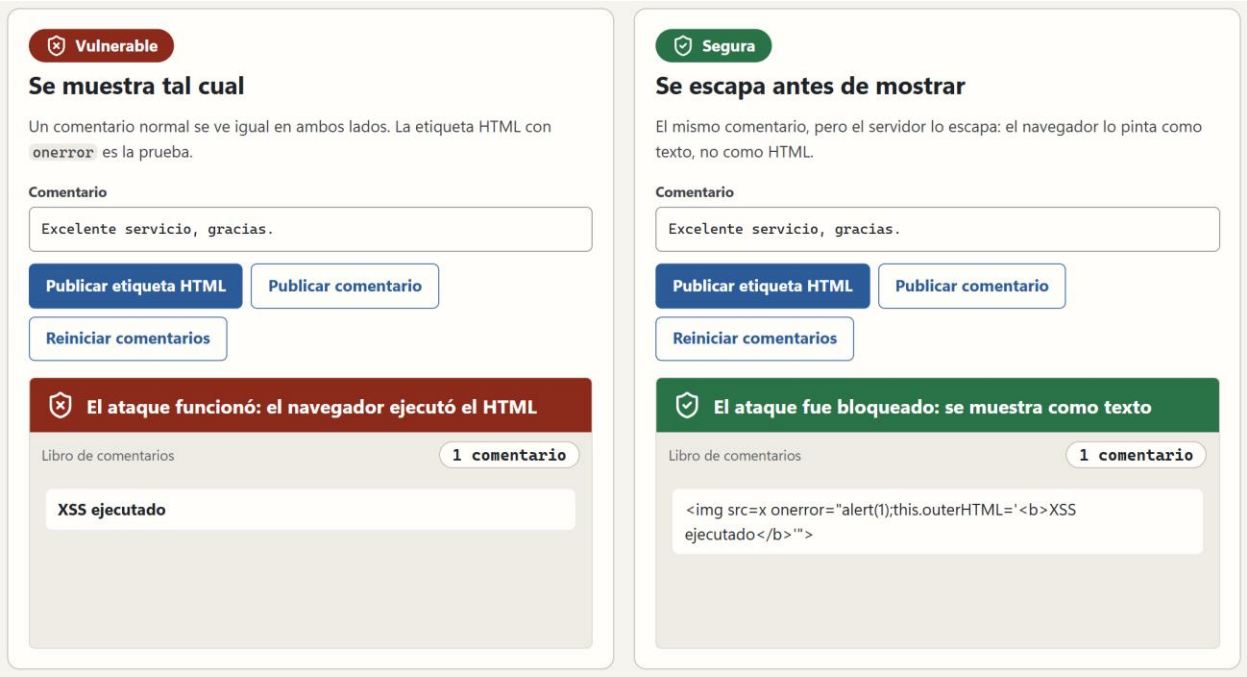
Paso 2. En la versión vulnerable, hacer clic en **Publicar etiqueta HTML**. El laboratorio publica un comentario con la etiqueta **<img src=x onerror=...>**, que lleva un manejador **onerror** con un **alert(1)** y una marca visible.

Paso 3. Observar que el navegador muestra un cuadro de alerta con el número 1 y que, en el libro de comentarios, la etiqueta fue reemplazada por el texto **XSS ejecutado**: interpretó el comentario como una etiqueta **<img>** real y ejecutó su manejador de error. El veredicto es «El ataque funcionó».

Paso 4. En la versión segura, hacer clic en **Publicar etiqueta HTML**. Observar que esta vez se ve el texto exacto de la etiqueta, como texto plano, sin alerta, y que el veredicto es «El ataque fue bloqueado».

Figura 6

Comparación: la etiqueta se ejecuta como HTML en la versión vulnerable (aparece XSS ejecutado) y queda como texto en la segura



Nota. Elaboración propia.

Resultados esperados

La versión vulnerable devuelve el comentario sin escapar y el navegador lo renderiza como HTML (salta el cuadro de alerta y aparece el texto XSS ejecutado, prueba de que la etiqueta **<img>** se creó de verdad). La versión segura devuelve el comentario con las entidades **< > "**, que el navegador muestra como el texto literal de la etiqueta.

Criterios de éxito

El estudiante logra que el navegador interprete una etiqueta HTML inyectada en la versión vulnerable, y muestra que la versión segura la neutraliza, mostrándola solo como texto.

Medidas de mitigación

Código de la versión segura (módulo **modules/a05.js**):

```
function escapeHtml(s) {
  return String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;')
    .replace(/>/g, '&gt;').replace(/"/g, '&quot;').replace(/'/g, '&#39;');
}
app.get('/a05/safe/comments', (_req, res) =>
  res.json({ comments: safeComments.map(escapeHtml) }));
```

Escapar siempre la salida que se inserta en HTML, en el punto donde se renderiza, no solo al guardar el dato. Evitar **innerHTML** con datos de usuario; preferir **textContent** o un motor de plantillas que escape por defecto (la mayoría de los frameworks modernos lo hacen). Añadir una Content Security Policy (CSP) como segunda capa de defensa frente a scripts inyectados.

Preguntas de análisis

1. ¿Por qué escapar la entrada al guardarla no basta, y hay que escapar también al mostrarla?

2. Si el comentario vulnerable lo pudiera leer otro usuario (no solo quien lo escribió), ¿qué podría hacer un script malicioso en su sesión (robar la cookie, actuar en su nombre)?

Taller 4. A07:2025 Authentication Failures (JWT sin verificar)

Nombre de la actividad

De invitado a administrador: falsificando un token sin tocar la clave secreta.

Vulnerabilidad asociada

A07:2025 Authentication Failures. OWASP describe aquí los fallos en la verificación de la identidad del usuario, incluyendo tokens de sesión que no se validan correctamente. El defecto de este taller corresponde a **CWE-287 Improper Authentication**: el servidor acepta como válida una afirmación de identidad (el contenido del token) sin comprobar su firma (OWASP Top 10:2025, A07).

Descripción del entorno

Módulo **modules/a07.js** del laboratorio: emite tokens con el mismo formato que un JWT (encabezado.carga.firma, codificados en Base64url y firmados con HMAC-SHA256). La ruta de perfil vulnerable decodifica la carga del token y confía en su contenido sin llamar nunca a la función que verifica la firma. La ruta de perfil segura recalcula la firma y compara con **crypto.timingSafeEqual** antes de confiar en el contenido.

Objetivo del reto

Obtener un token válido como usuario "invitado", modificar su carga para declararse "admin" sin volver a firmarlo (conservando la firma original, ahora inválida para el nuevo contenido), y comprobar que la ruta vulnerable acepta ese cambio mientras la ruta segura lo rechaza.

Instrucciones

Paso 1. Con el laboratorio corriendo, abrir la pestaña **A07** y hacer clic en **Obtener token de "invitado"**. El token aparece dividido en tres partes separadas por puntos y se copia automáticamente en los dos cuadros de texto de abajo.

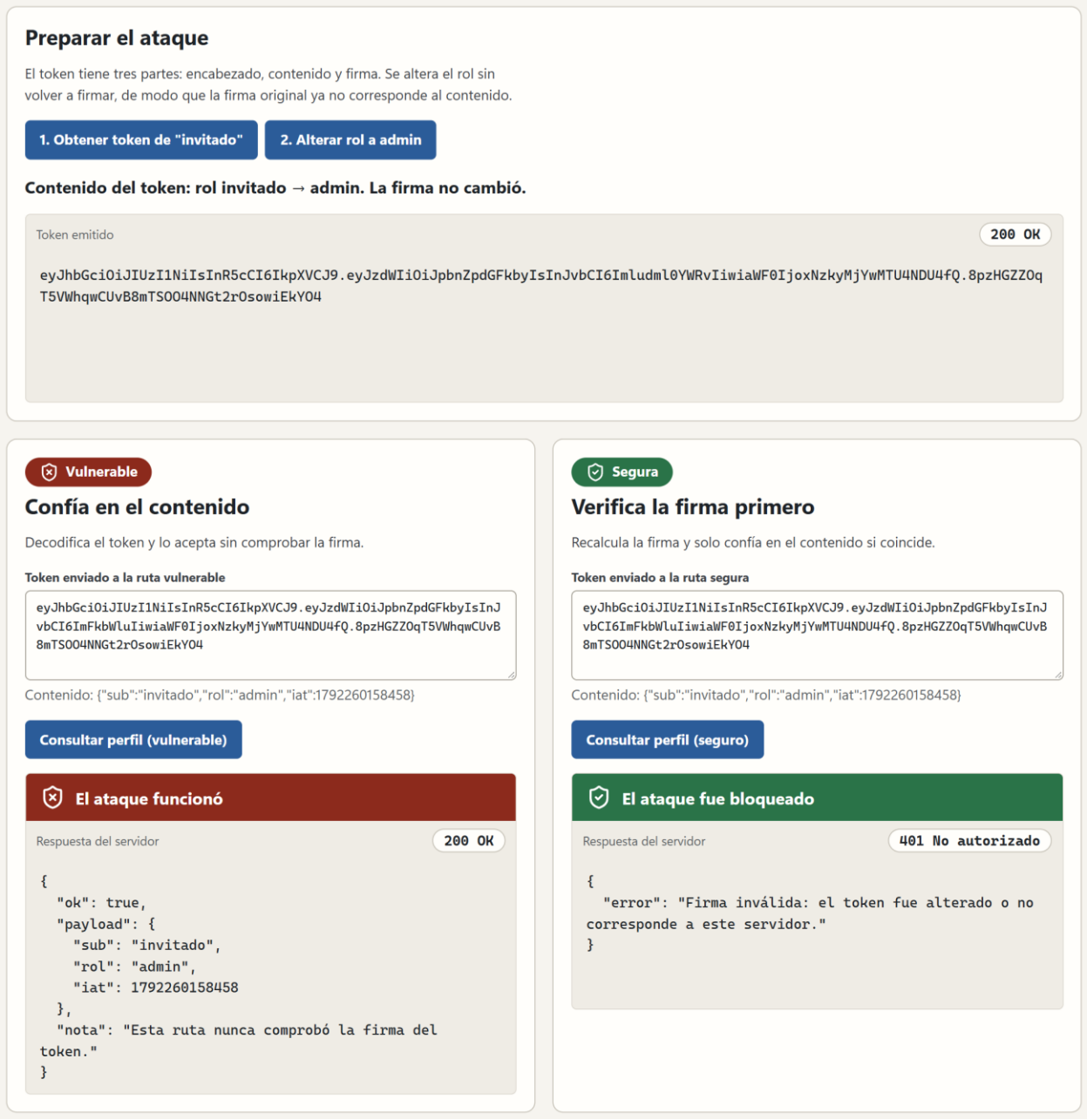
Paso 2. Hacer clic en **Alterar rol a admin**. El laboratorio decodifica en Base64url la parte central del token (la carga), cambia **"rol":"invitado"** por **"rol":"admin"**, la vuelve a codificar y deja el encabezado y la firma exactamente como estaban. Bajo cada cuadro se muestra el contenido ya decodificado.

Paso 3. Con el token alterado ya cargado en el cuadro de la versión vulnerable, hacer clic en **Consultar perfil (vulnerable)**. Observar que la respuesta es 200 OK e incluye **"rol": "admin"**, aunque el token real nunca fue firmado con ese contenido.

Paso 4. Con el mismo token en el cuadro de la versión segura, hacer clic en **Consultar perfil (seguro)**. Observar que la respuesta es 401 No autorizado, con un error de firma inválida.

Figura 7

Preparación del ataque y comparación: el perfil vulnerable acepta el rol admin falsificado; el seguro lo rechaza por firma inválida



Nota. Elaboración propia.

Resultados esperados

La ruta vulnerable devuelve **{"ok": true, "payload": {"rol": "admin", ...}}** para el token alterado. La ruta segura devuelve **{"error": "Firma inválida: el token fue alterado o no corresponde a este servidor."}** para ese mismo token.

Criterios de éxito

El estudiante logra que la ruta vulnerable confíe en un rol que el servidor nunca firmó, y demuestra que la ruta segura detecta la alteración y la rechaza.

Medidas de mitigación

Código de la versión segura (módulo **modules/a07.js**):

```
function verify(token) {
  try {
    const [h, b, s] = String(token).split('.');
    const esperada = crypto.createHmac('sha256', SECRET).update(h + '.' + b).digest();
    const dada = Buffer.from(s, 'base64url');
    if (dada.length !== esperada.length || !crypto.timingSafeEqual(esperada, dada))
return null;
    return JSON.parse(Buffer.from(b, 'base64url').toString('utf8'));
  } catch { return null; }
}
app.get('/a07/safe/perfil', (req, res) => {
  const payload = verify(token);
  if (!payload) return res.status(401).json({ error: 'Firma invalida.' });
  res.json({ ok: true, payload });
});
```

Verificar siempre la firma del token con una librería de JWT madura antes de leer su contenido; nunca decodificar y confiar en la carga sin ese paso. Usar **timingSafeEqual** (comparación en tiempo constante) para comparar firmas, nunca ===, y evitar que el servidor acepte el algoritmo "none". Mantener la clave de firma fuera del código fuente, en una variable de entorno o un gestor de secretos, y usar tokens de vida corta.

Preguntas de análisis

1. ¿Por qué firmar el token no sirve de nada si el servidor nunca comprueba esa firma al leerlo? 2. Si un atacante no conoce la clave secreta del servidor, ¿por qué de todos modos logra que el servidor acepte un rol distinto al que firmó originalmente?

Resumen de los cuatro talleres

Tabla 1
Vulnerabilidades trabajadas en la Guía 6

Taller	Categoría OWASP Top 10:2025	CWE principal	Módulo del laboratorio
1	A01 Broken Access Control (SSRF)	CWE-918	modules/a01.js
2	A02 Security Misconfiguration	CWE-16	modules/a02.js
3	A05 Injection (XSS almacenado)	CWE-79	modules/a05.js
4	A07 Authentication Failures (JWT)	CWE-287	modules/a07.js

Nota. Elaboración propia.

Anexo: cómo ejecutar el laboratorio

Requisitos: Node.js 18 o superior. Dentro de la carpeta **lab/** de este entregable, ejecutar:

```
npm install
npm start
```

El servidor queda disponible en **http://127.0.0.1:3000** (o el puerto que indique la variable de entorno **PORT**). La interfaz tiene una pestaña por cada módulo (A01, A02, A05, A07); cada una incluye un bloque **vulnerable** y uno **segura** para repetir en vivo lo mostrado en este documento.

Referencias

OWASP. (2025a). *A01:2025 Broken Access Control.* OWASP Top 10:2025. https://top10.owasp.org/2025/A01_2025-Broken_Access_Control/

OWASP. (2025b). *A02:2025 Security Misconfiguration.* OWASP Top 10:2025. https://top10.owasp.org/2025/A02_2025-Security_Misconfiguration/

OWASP. (2025c). *A05:2025 Injection.* OWASP Top 10:2025. https://top10.owasp.org/2025/A05_2025-Injection/

OWASP. (2025d). *A07:2025 Authentication Failures.* OWASP Top 10:2025. https://top10.owasp.org/2025/A07_2025-Authentication_Failures/